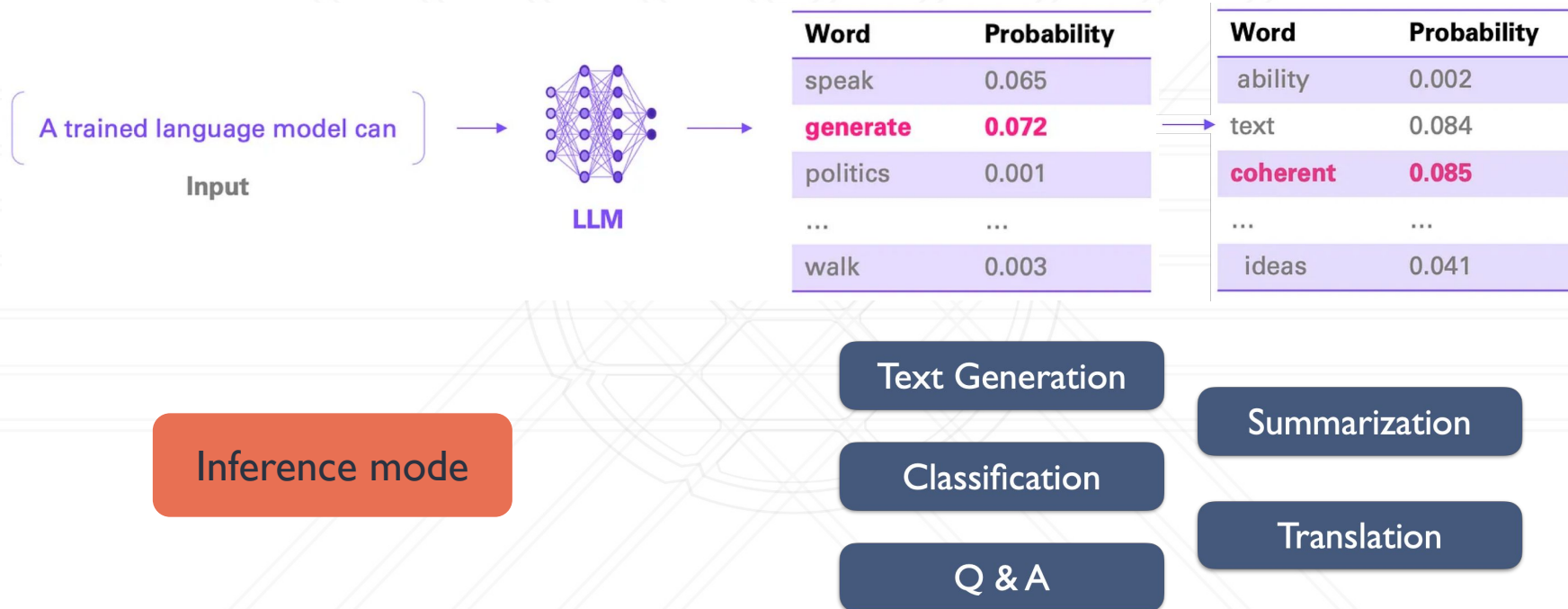


# Distributed Deep Learning on GPU-based Clusters

Prajwal Singhanian, Lannie Dalton Hough, Abhinav Bhatele  
Department of Computer Science

# How are neural networks used?



# Get the tutorial repository

---

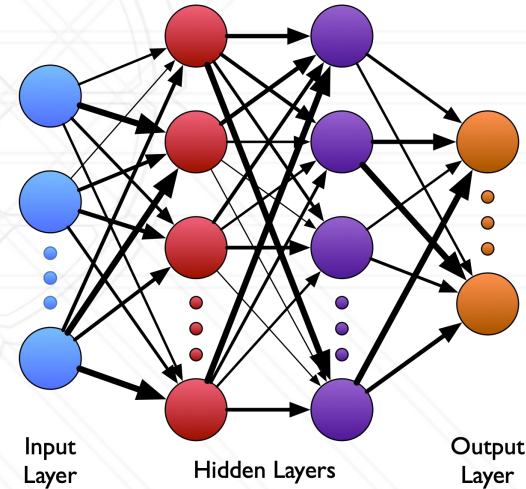
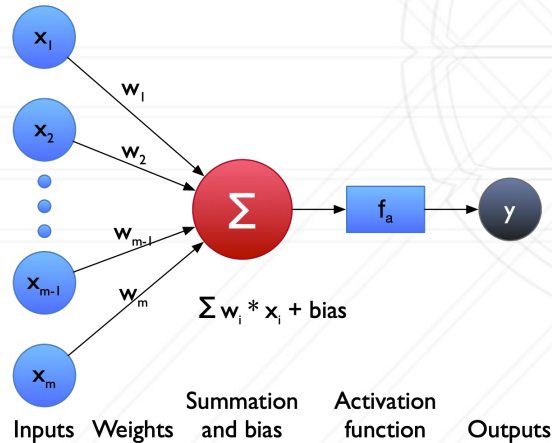
- Clone the git repository as follows:

```
ssh <username>@login.zaratan.umd.edu  
git clone https://github.com/axonn-ai/distrib-dl-tutorial.git
```

# What are neural networks?

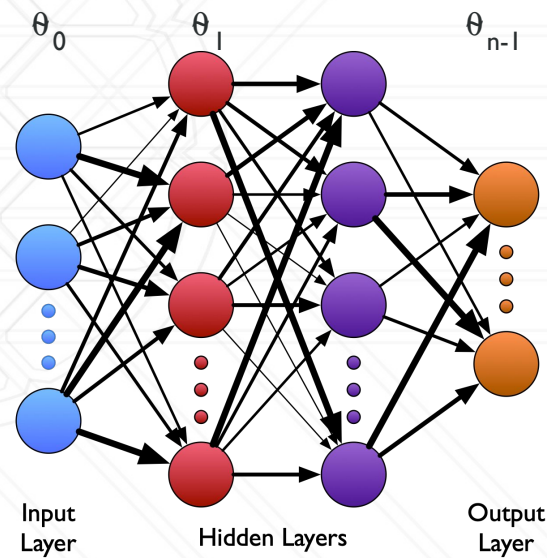
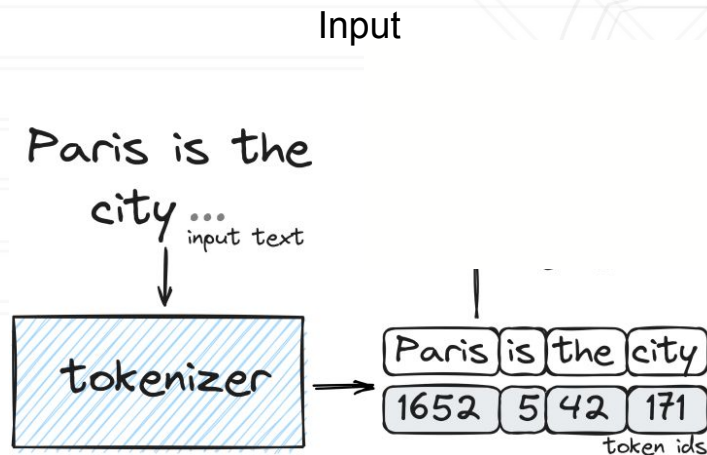
Neural networks are “parameterized” function approximators that can be used to model complex functions

Several layers that process input data



# How is a neural network trained?

Problem: Find a set of weights/parameters that best fits the function we are trying to learn over a given training dataset



# Other terms and definitions

---

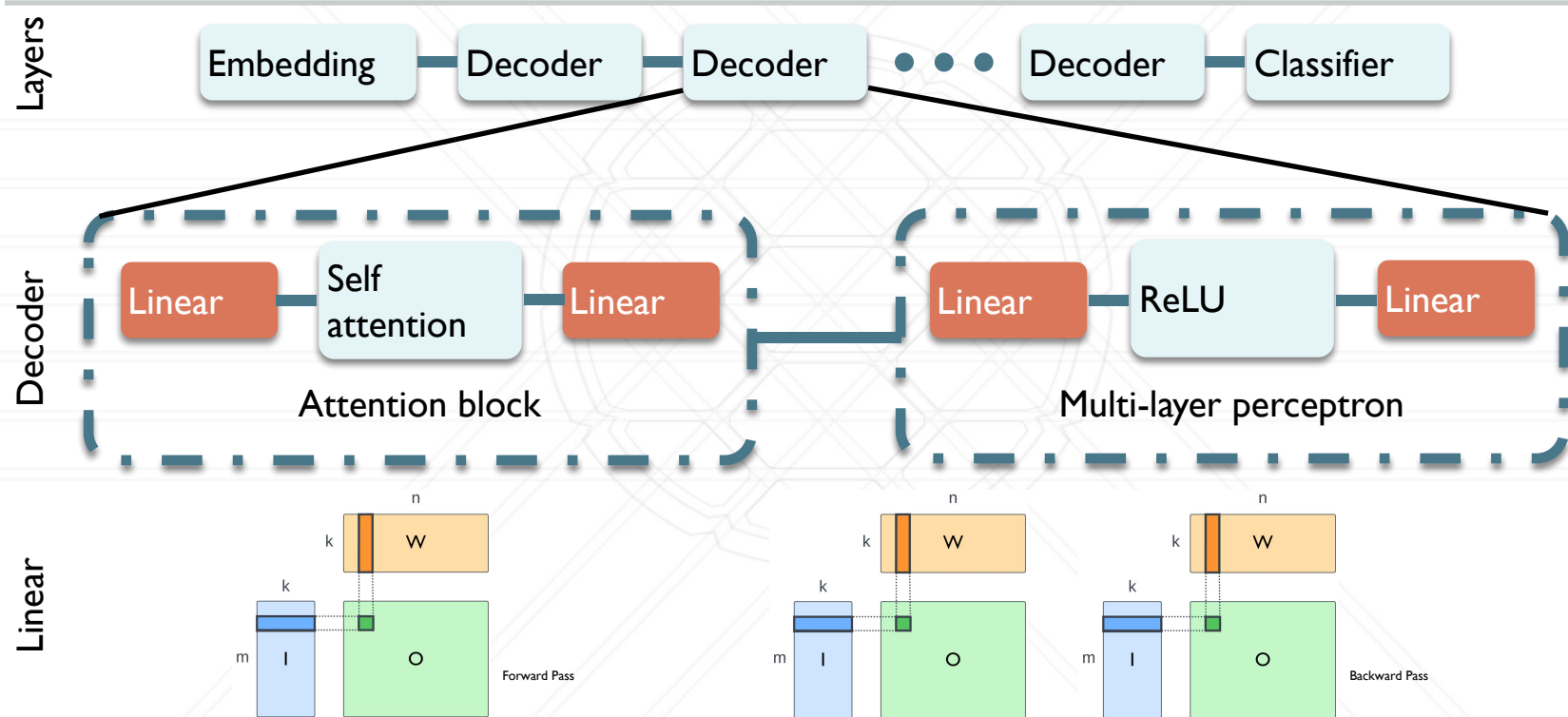
- **Loss:** a scalar proxy that when minimized leads to higher accuracy
- **Learning/training:** task of selecting weights that lead to an accurate function / minimizes the loss
- **Gradient descent:** process of updating the weights using gradients (derivatives) of the loss weighted by a learning rate
- **Batch:** Small subsets of the dataset processed independently
- **Epoch:** One pass over all the batches
  
- **Fine-tuning:** updating weights for a specific task
- **Inference:** using a trained or fine-tuned model to generate output

# Stochastic Gradient Descent

---

```
while (remaining_batches) {  
    Read a single batch  
  
    Forward pass: perform matrix multiplies to compute  
        output activations, and a loss on the batch  
  
    Backward pass: matrix multiplies to compute gradients of  
        the loss w.r.t. parameters via backpropagation  
  
    Optimizer step: use gradients to update the weights or  
        parameters such that loss is gradually reduced  
}
```

# Why is LLM training well-suited for HPC?





# PyTorch



- torch – a Python library for tensor computations with GPU support
- torch.nn – implementations of various neural network layers

# PyTorch Lightning



- High-level API as an alternative to using PyTorch directly
  - Automatic Mixed Precision
  - Vendor Agnostic - CPU, GPU, TPU
  - **Seamlessly switch between parallel strategies**
- We will start with looking at single GPU training using PyTorch Lightning

# Fine-tuning LLMs

---

- Fine-tuning is a popular approach to tune pre-trained LLMs for specific tasks
- Instruction fine-tuning (IFT): Train on datasets of [instruction, output] pairs.
- Example use case: build a chatbot by tuning an LLM to follow instructions.

# Samples from an IFT dataset

| instruction                                                                       | output                                                                                                                                                                                                    |
|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| string · lengths                                                                  | string · lengths                                                                                                                                                                                          |
|  |                                                                                                                         |
| Give three tips for staying healthy.                                              | 1. Eat a balanced diet and make sure to include plenty of fruits and vegetables. 2. Exercise regularly to keep your body active and strong. 3. Get enough sleep and maintain a consistent sleep schedule. |
| What are the three primary colors?                                                | The three primary colors are red, blue, and yellow.                                                                                                                                                       |

These are from the **alpaca** dataset, which we will be using in the tutorial

# Using PyTorch Lightning

---

- Code location in the tutorial repo: train.py

| Parameter         | Value             |
|-------------------|-------------------|
| model_id          | microsoft/phi-1_5 |
| dataset_id        | alpaca            |
| strategy          | single-gpu        |
| global_batch_size | 32                |

```
CONFIG_FILE=configs/single_gpu.json sbatch train_single.sh
```

```
squeue --me  
ls -lrth
```

# Mixed-precision Training

- GPUs have FP32, FP64 and tensor cores
- We can optimize performance by doing some operations in lower precision (16-bit)



# Mixed-precision Training

---

- Code location in the tutorial repo: `train.py`
- Open `configs/single_gpu.json` and change precision to `bf16-mixed`

```
CONFIG_FILE=configs/single_gpu.json sbatch train_single.sh
```

# Activation Checkpointing

---

- Activations are outputs of individual layers
- To save memory, we checkpoint only inputs to each layer
  - Regenerate intermediate and output activations as needed in the backward pass
- Code location in the tutorial repo: `train.py`

```
CHECKPOINT_ACTIVATIONS=true sbatch --reservation=sc2024  
run_sequential.sh
```



# How to download a different model?

---

```
cd external; python download.py <model-name>
```

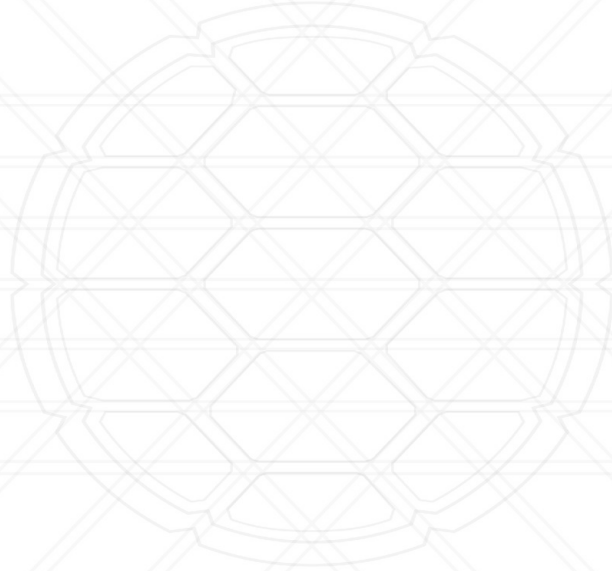
To get a list of available models in LitGPT -

```
python download.py list
```

# Where does a model file come from?

---

- Write your own
- Get a pre-implemented model from somewhere (huggingface, LitGPT, ...)



**Prajwal Singhania, Lannie Dalton Hough, Abhinav Bhatele**

Department of Computer Science

[prajwal@umd.edu](mailto:prajwal@umd.edu), [bhatele@cs.umd.edu](mailto:bhatele@cs.umd.edu), [ldhough@umd.edu](mailto:ldhough@umd.edu)

